1 Introduction and Methodology

The objective of this task was to implement the Gradient Descent (GD) algorithm and evaluate it on two benchmark functions: Sphere and Matyas. To determine if more complex optimizers offer advantages on these simple landscapes, GD with Momentum, AdaGrad, and Adam were also implemented.

Gradients were calculated numerically using the central difference scheme. A step size of $h = 10^{-5}$ was selected as an optimal compromise that prevents losing precision in floating-point operations while minimizing truncation error. Coordinate clipping (e.g., `np.clip`) was intentionally omitted. Instead, the loop terminates if $f(x) > 10^{15}$. This allows for the observation of authentic numerical divergence on the charts rather than artificial bounding.

2 Learning Rate Analysis (Starting point [10.0, -10.0])

2.1 Sphere Function

Learning rates (lr) [0.01, 0.05, 0.1, 0.5, 1.0, 1.1] were tested with a limit of 100 iterations.

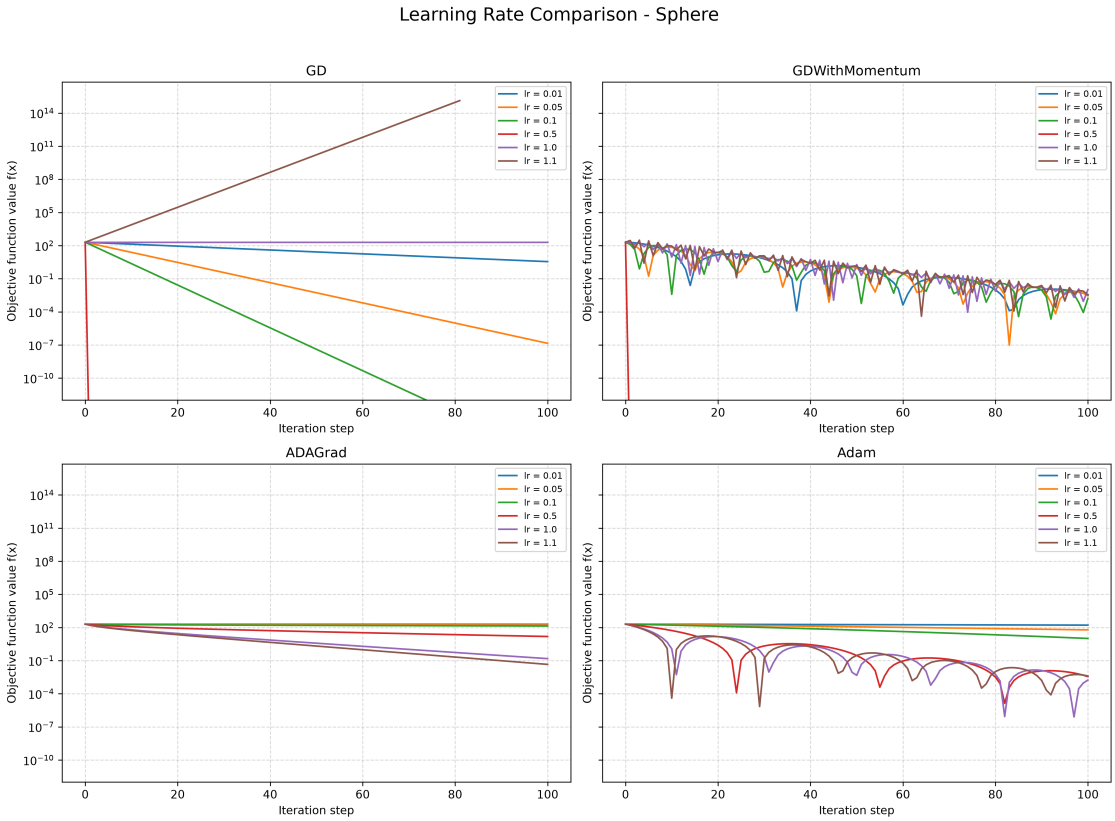


Figure 1: Convergence comparison across various learning rates on the Sphere function.

Table 1: Iterations to reach $\|\nabla f(x)\| < 10^{-6}$ on the Sphere Function

Optimizer	$lr = 0.01$	$lr = 0.05$	$lr = 0.1$	$lr = 0.5$	$lr = 1.0$	$lr = 1.1$
GD	> 100	> 100	77	1	No conv. (Osc.)	Diverged (Expl.)
GD with Momentum	> 100	> 100	> 100	1	> 100 (Osc.)	> 100 (Osc.)
AdaGrad	> 100	> 100	> 100	> 100	> 100	> 100
Adam	> 100	> 100	> 100	> 100	> 100	> 100

Gradient Descent (GD): Small learning rates (0.01, 0.05) converge too slowly to reach the 10^{-6}

gradient norm threshold within 100 iterations. A step of $lr = 0.1$ provides smooth convergence in 77 iterations. For the Sphere function ($\nabla f = [2x_1, 2x_2]$), the update rule simplifies to $x_{k+1} = x_k(1 - 2 \cdot lr)$, so $lr = 0.5$ reaches $(0, 0)$ in just 1 step. For $lr = 1.0$, GD jumps endlessly between $[10, -10]$ and $[-10, 10]$ ($f(x) = 200$), while $lr = 1.1$ diverges above 10^{15} in 81 iterations.

Alternative Optimizers: AdaGrad slows down quickly because adding squared gradients ($\sum \nabla f^2$) in the denominator continuously reduces the effective step size, requiring larger learning rates or more iterations. GD with Momentum and Adam build up speed via their momentum terms ($\beta = 0.9$), causing them to overshoot $(0, 0)$ and oscillate. Notably, for $lr \in \{1.0, 1.1\}$, the $\beta = 0.9$ factor dampens the updates in Momentum, allowing $f(x)$ to slowly drop toward 10^{-2} instead of exploding like standard GD.

2.2 Matyas Function

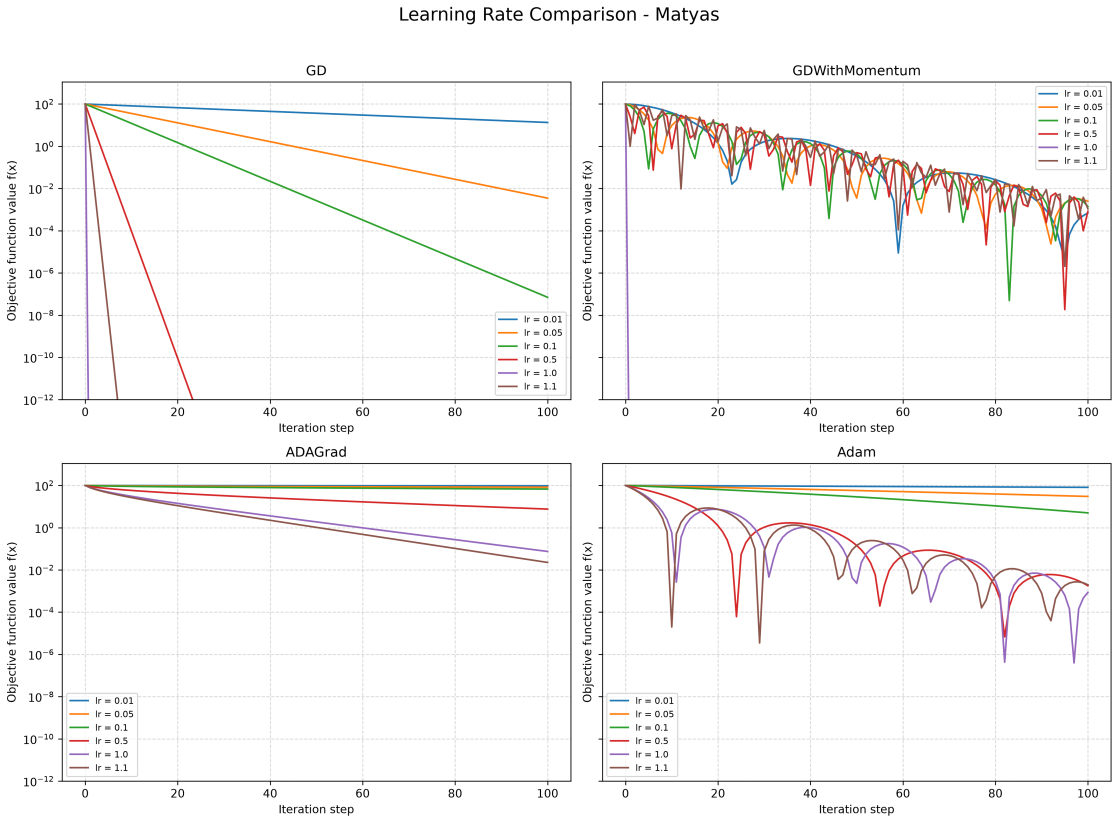


Figure 2: Convergence comparison across various learning rates on the Matyas function.

Table 2: Iterations to reach $\|\nabla f(x)\| < 10^{-6}$ on the Matyas Function

Optimizer	$lr = 0.01$	$lr = 0.05$	$lr = 0.1$	$lr = 0.5$	$lr = 1.0$	$lr = 1.1$
GD	> 100	> 100	> 100	24	1	8
GD with Momentum	> 100	> 100	> 100	> 100	1	> 100
AdaGrad	> 100	> 100	> 100	> 100	> 100	> 100
Adam	> 100	> 100	> 100	> 100	> 100	> 100

Gradient Descent (GD): At $[10.0, -10.0]$, the gradient of the Matyas function equals $[10.0, -10.0]$, which is identical to the coordinates of the point itself. Because of this symmetry along $x_2 = -x_1$, the update rule becomes $x_{k+1} = x_k(1 - lr)$. Consequently, $lr = 1.0$ hits $(0, 0)$ in 1 step, $lr = 1.1$ converges in 8 steps, and $lr = 0.5$ takes 24 steps. Meanwhile, $lr = 0.1$ drops smoothly to $f(x) \approx 10^{-7}$ at step 100, but needs more than 100 iterations to reach $\|\nabla f(x)\| < 10^{-6}$.

Alternative Optimizers: AdaGrad again progresses too slowly for $lr \leq 1.1$. Both GD with Momentum and Adam show strong oscillations at higher learning rates due to momentum ($\beta_1 = 0.9$). In Adam, the

second moment ($\beta_2 = 0.999$) remembers the large initial squared gradients, which gradually reduces the step size near the minimum, while the first moment ($\beta_1 = 0.9$) still pushes the point past $(0, 0)$, creating the sharp dips visible on the log-scale plot.

3 Trajectory Analysis ($lr = 0.1$, limit 1000 iterations)

3.1 Sphere Function (Symmetrical Bowl)

The Sphere function forms a symmetrical bowl where the gradient $\nabla f = [2x_1, 2x_2]$ always points straight toward $(0, 0)$. However, adaptive scaling changes the path from the asymmetrical start $[-7.0, 2.0]$:

- **Gradient Descent (GD):** Moves in a straight line to the minimum from all starting points, as the step on each axis is directly proportional to the distance from $(0, 0)$.
- **GD with Momentum:** Also moves along a straight line in the (X_1, X_2) plane because both coordinates are updated with the exact same proportions (the curved look in 3D is just the vertical shape of the bowl). However, built-up velocity makes it overshoot $(0, 0)$ and oscillate back and forth along this line.
- **Adam:** Follows a curved path from $[-7.0, 2.0]$. Because Adam scales each coordinate separately using $\hat{m}_i/\sqrt{\hat{v}_i}$, it starts by taking similar steps of roughly lr on both axes. As a result, it reaches zero on the shorter X_2 axis first, forcing the trajectory to bend and finish along the X_1 axis.
- **AdaGrad:** Also curves from $[-7.0, 2.0]$ because it normalizes each coordinate separately ($\nabla f_i/\sqrt{\sum \nabla f_i^2}$). However, the growing sum in the denominator reduces the step size too quickly, stopping the algorithm before it reaches the minimum within 1000 steps.

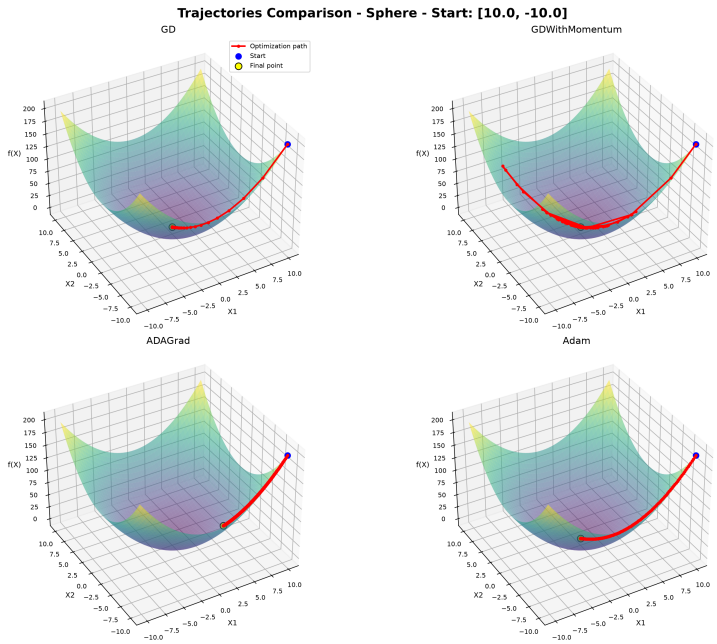


Figure 3: Trajectories from the start point $[10.0, -10.0]$.

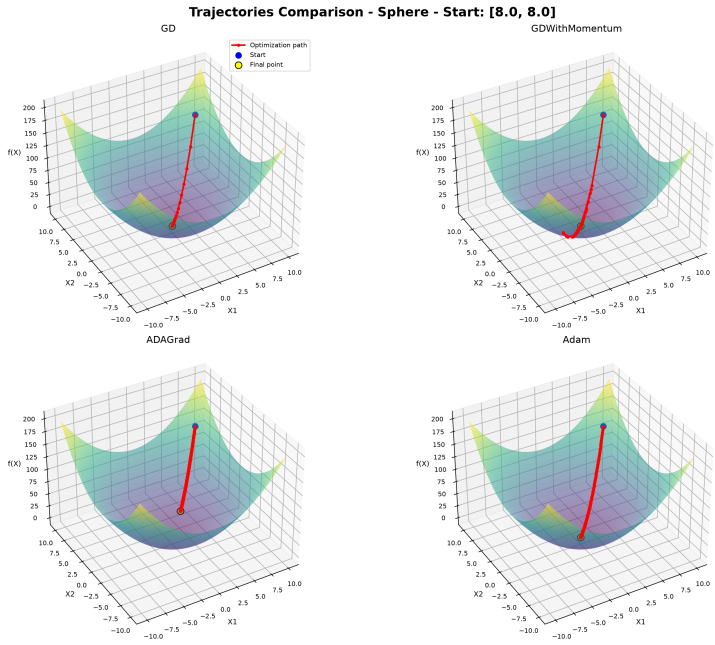


Figure 4: Trajectories from the diagonal start point $[8.0, 8.0]$.

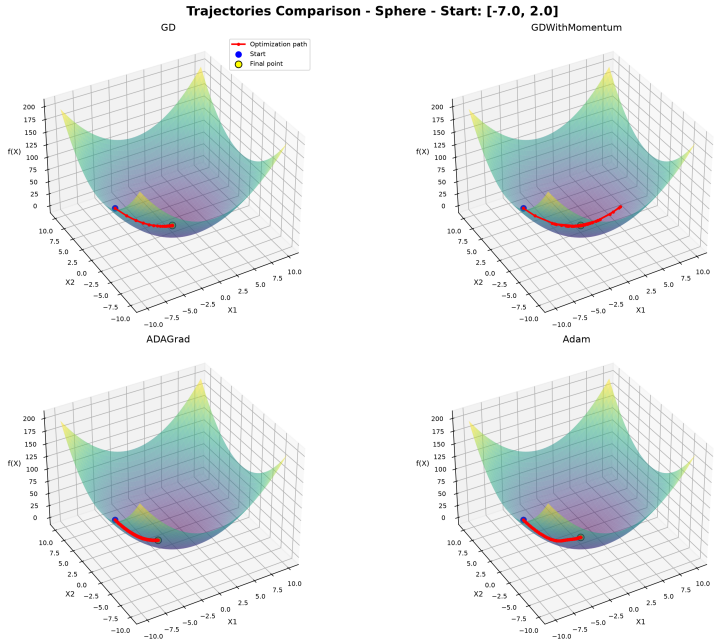


Figure 5: Trajectories from the asymmetrical start point $[-7.0, 2.0]$.

3.2 Matyas Function (Narrow Valley)

The $-0.48x_1x_2$ term shapes the landscape into a narrow valley along the diagonal $x_1 = x_2$, with steep side walls and a nearly flat bottom. As a result, the gradient on the walls points toward the valley floor rather than directly toward $(0, 0)$:

- **Gradient Descent (GD):** Moves in a straight line from the symmetrical points $[10.0, -10.0]$ and $[8.0, 8.0]$. From $[-7.0, 2.0]$, GD follows a curved path: the steep side wall first pulls the point quickly down to the valley floor ($x_1 \approx x_2$), and once the wall slope flattens out, the path turns to slowly follow the gentle slope along the bottom toward $(0, 0)$.
- **GD with Momentum:** Gains high speed on the steep walls, strongly overshooting $(0, 0)$ from $[10.0, -10.0]$ and swinging across the floor onto the opposite wall from $[-7.0, 2.0]$. From $[8.0, 8.0]$, the slope along the bottom is very flat (initial gradient $[0.32, 0.32]$), so it never builds enough speed to overshoot and moves straight.
- **Adam:** Dividing by $\sqrt{\hat{v}_t}$ limits the initial step size to about $lr = 0.1$, which prevents large overshooting on the 3D plot from $[10.0, -10.0]$. From $[-7.0, 2.0]$, independent axis scaling guides it

diagonally to the valley floor before making a sharp turn toward the origin.

- **AdaGrad:** Moves partway down the wall from $[10.0, -10.0]$ and $[-7.0, 2.0]$, and partway along the bottom from $[8.0, 8.0]$, before the accumulated squared gradients shrink the step size too much to finish within 1000 iterations at $lr = 0.1$.

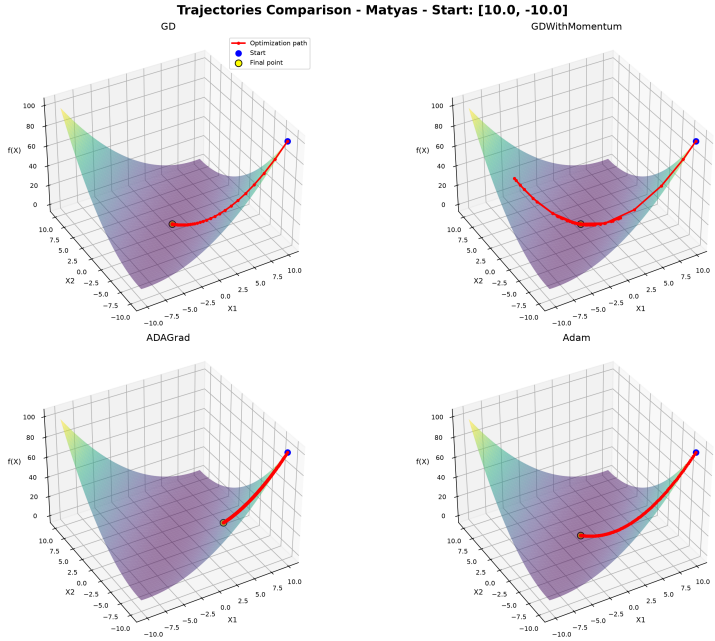


Figure 6: Trajectories from $[10.0, -10.0]$, showing symmetric descent down the steep wall.

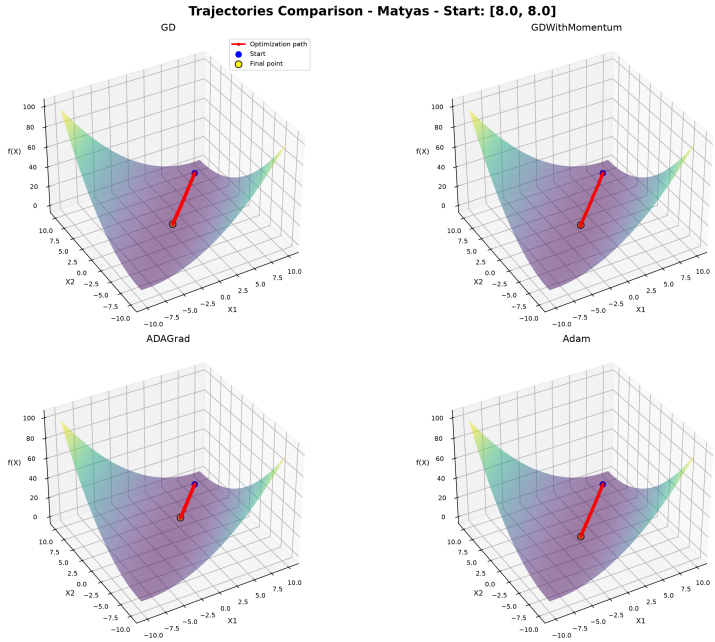


Figure 7: Trajectories from $[8.0, 8.0]$, showing straight descent along the valley floor.

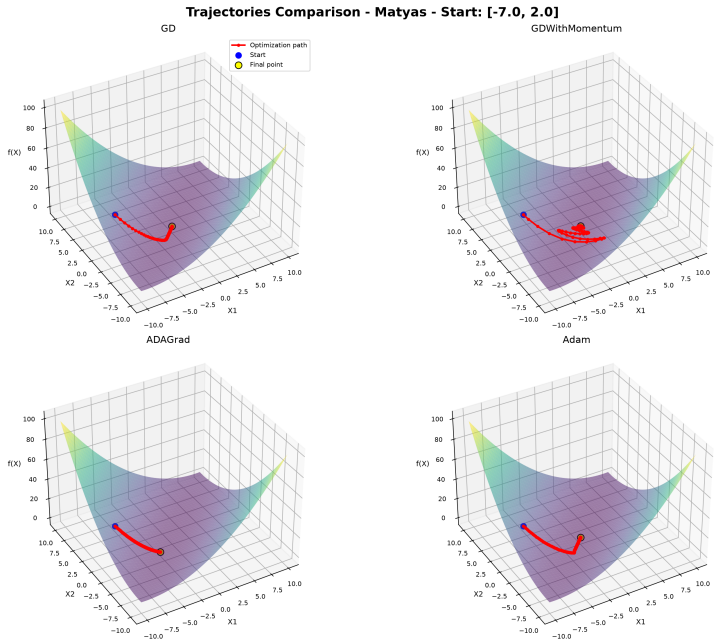


Figure 8: Trajectories from $[-7.0, 2.0]$, showing the curved descent into the valley.

Declaration of LLM Usage

A Large Language Model (Gemini 3.1 Pro) was utilized during the preparation of this report. Specifically, the model assisted in drafting and translating the text into academic English, configuring the LaTeX formatting and layout, consulting on the numerical step size h for the finite difference scheme, and implementing the visualization functions (`plot_all_optimizers_grid` and `generate_trajectory_3d_reports`). The core optimization algorithms, experimental execution, numerical results, and interpretations remain the original work of the author.